IT4931 / IT5384 / IT4043E · SOICT · HUST

Hadoop Ecosystem

Lecture 2 — Hadoop ecosystem

HDFS · MapReduce · YARN · ecosystem tools · cloud

Learning outcomes (CLO1, CLO2)

1. **Explain** why we still teach Hadoop (history + principles → L03/L07)
2. **Describe** core pieces: **HDFS**, **MapReduce**, **YARN**
3. **Walk through** Map → Shuffle → Reduce with a concrete example
4. **Place** ecosystem tools (Hive, Pig, HBase, Sqoop, Oozie, ZooKeeper, Kafka)
5. **Relate** Hadoop-on-cloud (EMR / managed Spark + object storage) to the same principles

Roadmap today

1. **2.1 Introduction** — goals, scale-out, fault tolerance, 2026 framing
2. **2.2 Components** — HDFS, MapReduce, YARN, ecosystem
3. **2.3 On clouds** — managed platforms; link to later lectures

2.1 Introduction

Agenda inside the chapter

- Apache Hadoop overview
- HDFS (preview — depth in **L03**)
- MapReduce programming model
- Other ecosystem components
- YARN & the big picture

Goals of Hadoop

Goal	Meaning
Scale-out storage	Grow capacity by adding nodes (commodity disks)
Scale-out compute	Parallel jobs across the cluster
Fault tolerance	Failures are normal — replicate data, restart tasks
Simple programming model	MapReduce (key, value) so apps ignore most distributed plumbing

Inspired by Google's early papers (GFS / MapReduce) — open-sourced as **Apache Hadoop**.

What Apache Hadoop is

- **Scalable, cost-effective** storage & processing
- Simple distributed programming model (**MapReduce**)
- **Scale-out**: add commodity servers (nodes)
- Designed for **hardware failures as normal**
- Inspired by Google's early data architecture (GFS / MapReduce papers)

Core Hadoop stack

Piece	Role
HDFS	Distributed storage
MapReduce	Batch processing framework (classic)
Hadoop Common	Shared libraries/utilities
YARN	Cluster resource management & scheduling (Hadoop 2+)

Scale-out by design

- Cluster of **nodes** — each participates in storage **and** compute
- Scale by adding machines (tens of thousands in large deployments)
- Default assumption: **distribute first**, not “one big box”

Fault tolerance via redundancy

- HDFS: files split into blocks, **replicated** (typically 3×)
- Failed node → data re-replicated; apps keep running
- MapReduce: jobs = many **independent tasks**; failed tasks **reschedule**
- Errors handled **automatically** below the application

Why we still teach Hadoop (2026)

Reason	Why it matters
Origin	2006–2015 foundation of open-source big data
Principles	Distribute data, move compute to data, tolerate failure
Legacy	Still appears in enterprises and teaching environments
Course path	L02 → **L03 HDFS** → **L07 Spark** (same ideas, better engines)

Hadoop 2026 — remains / lost ground

Still useful	Mostly replaced / evolved
HDFS concepts; some on-prem lakes	MapReduce as primary API → **Spark**
YARN ideas (resource isolation)	Many clouds use K8s / managed schedulers
Hive / HBase in niches	Trino/Presto, lakehouse tables, cloud warehouses
Teaching distributed thinking	“Install Hadoop locally” as default career skill

2.2 Hadoop components

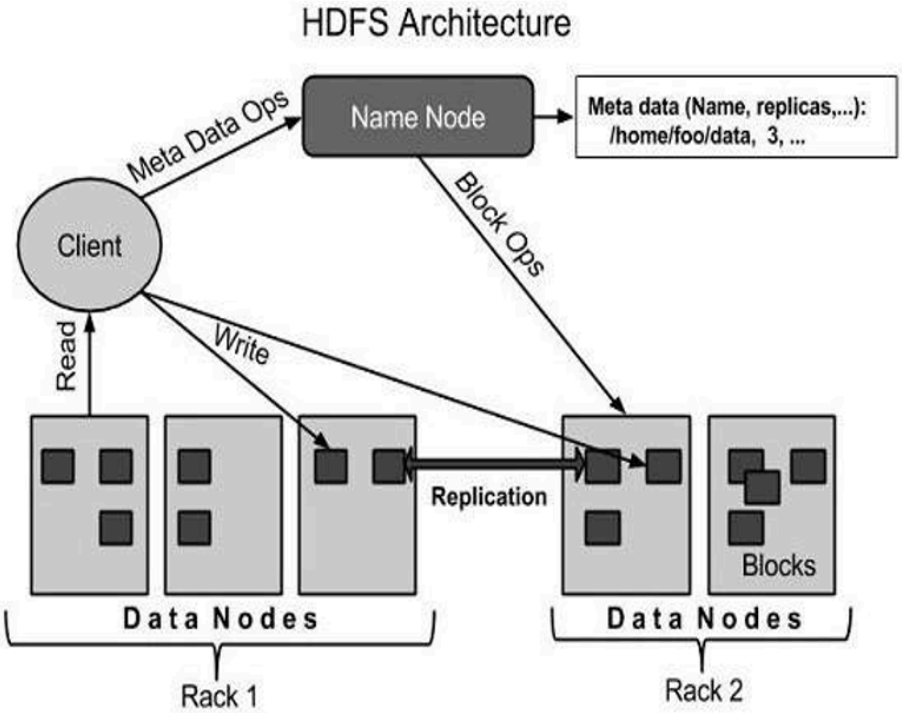
HDFS at a glance

- Reliable, cost-effective storage for **large files** (hundreds of MB → TB)
- Hierarchical paths like Unix (`/hust/soict/hello.txt`)
- Permissions inspired by Unix
- **Write-once / append / read-many** — not a general POSIX FS

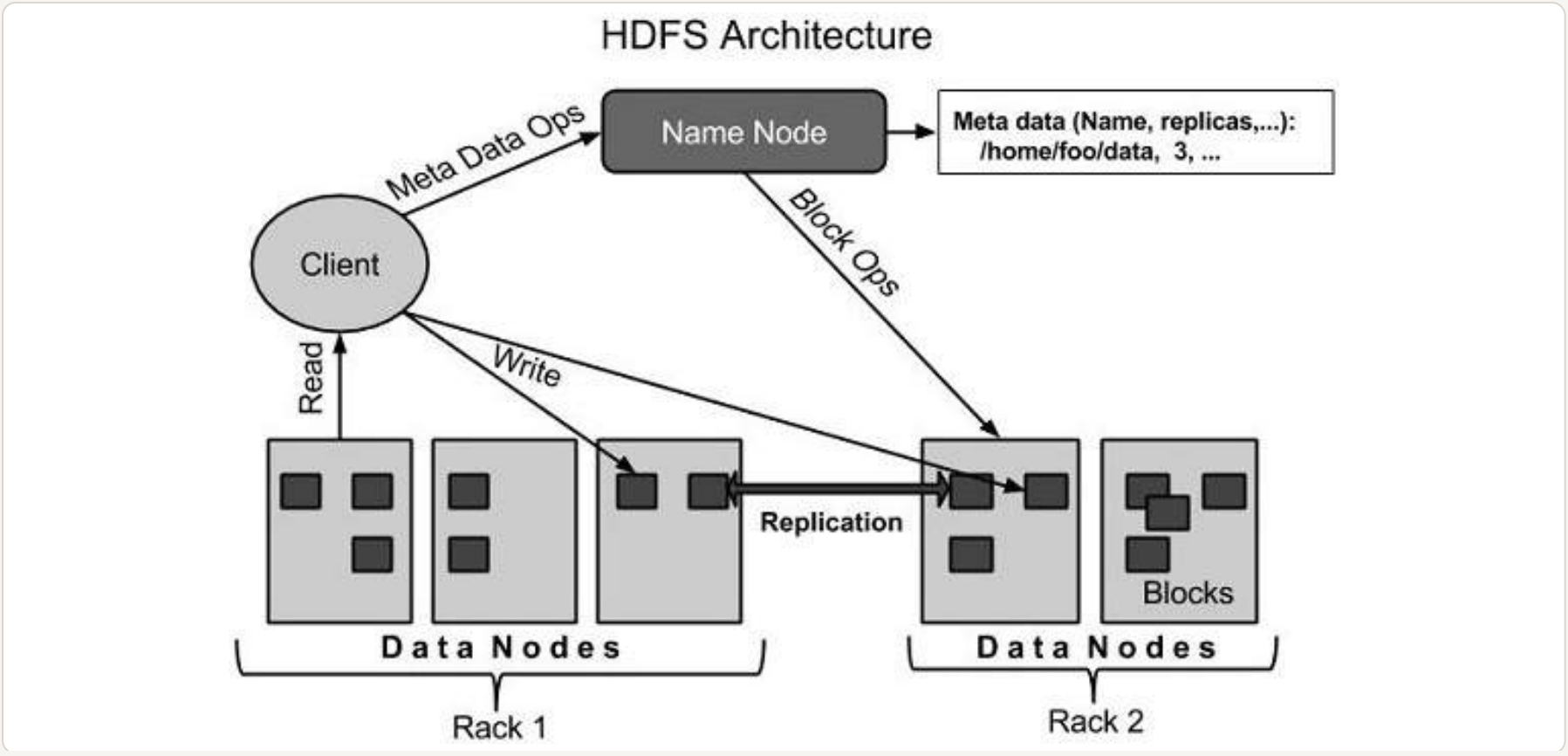
Depth: Lecture 3.

HDFS architecture

Master/slave: **NameNode** (namespace + block map) · **DataNodes** (block I/O).



HDFS architecture (alternate view)

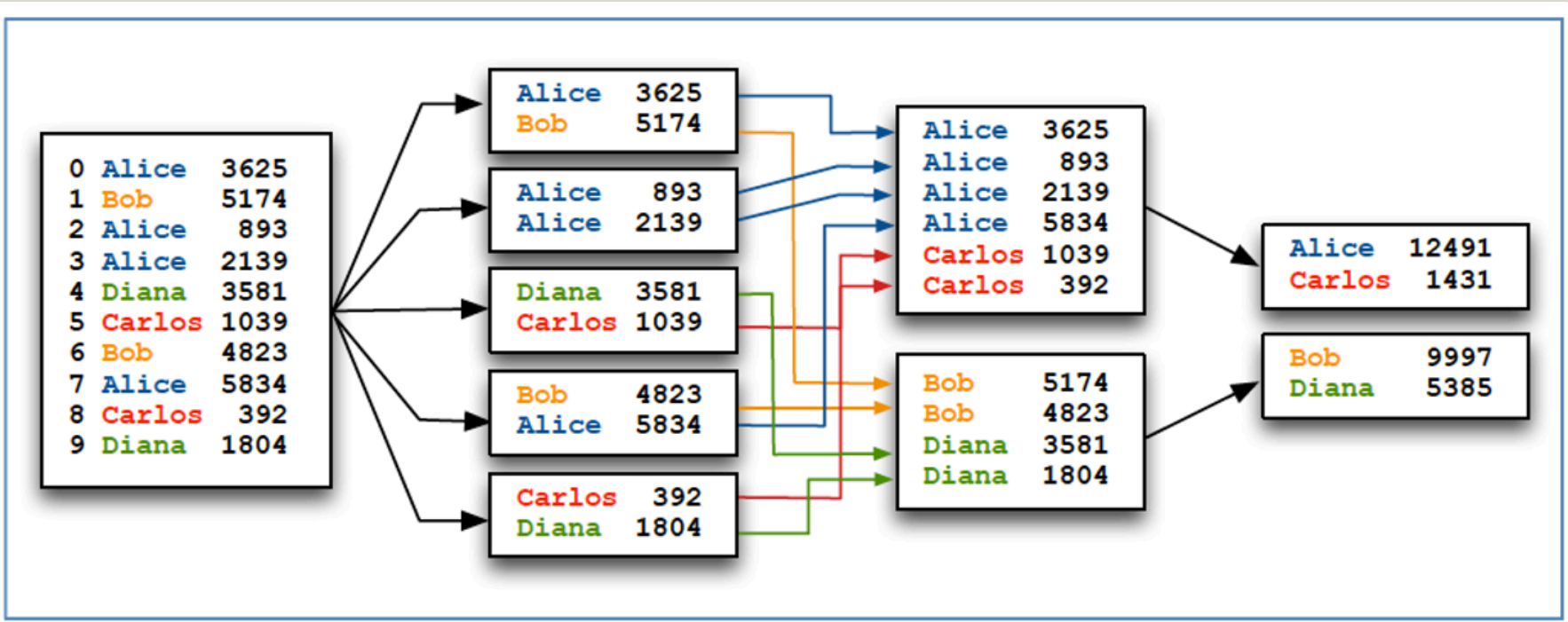


HDFS design principles

- **I/O pattern:** append-friendly → simpler concurrency
- **Large blocks** (historically 64–128 MB+) → less metadata, better sequential throughput
- **Replication** for durability
- DataNode failure → re-replication; NameNode recovery patterns (Secondary NN / HA in modern setups)

MapReduce model

Not a language — a **programming model**: simplicity, flexibility, scalability (Google → Hadoop).



A job = many isolated tasks

- One MapReduce **job** → many independent **tasks** across nodes
- Independence ⇒ scale + less sync between tasks
- Prefer **ship code to data** already on HDFS (data locality)

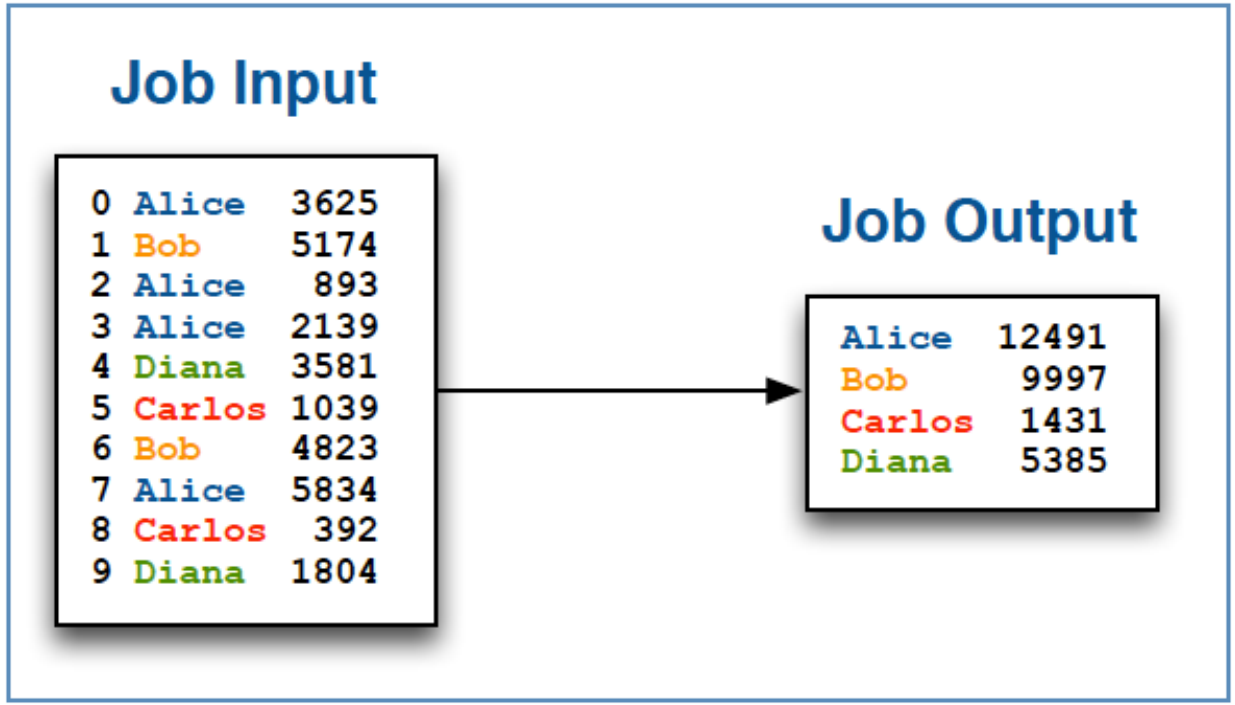
Map & Reduce as (key, value) transforms

```
Input splits  →  Map  →  (K', V') intermediate
              →  Shuffle/Sort by key
              →  Reduce  →  (K'', V'') final
```

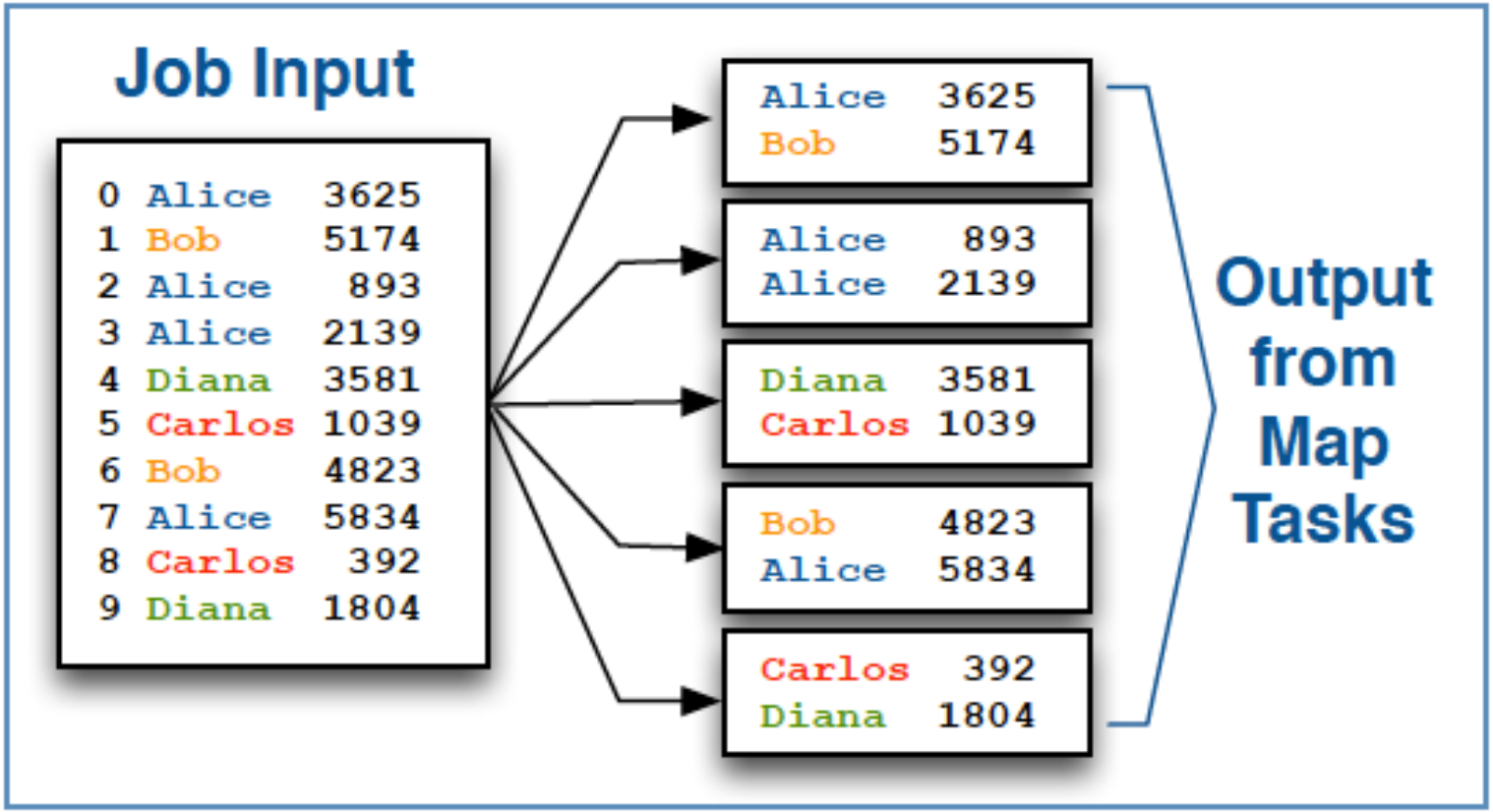
You implement **map** and **reduce**; the framework handles distribution, sort, and retries.

Example: sales by employee

Input: order lines · Output: total sales per employee.

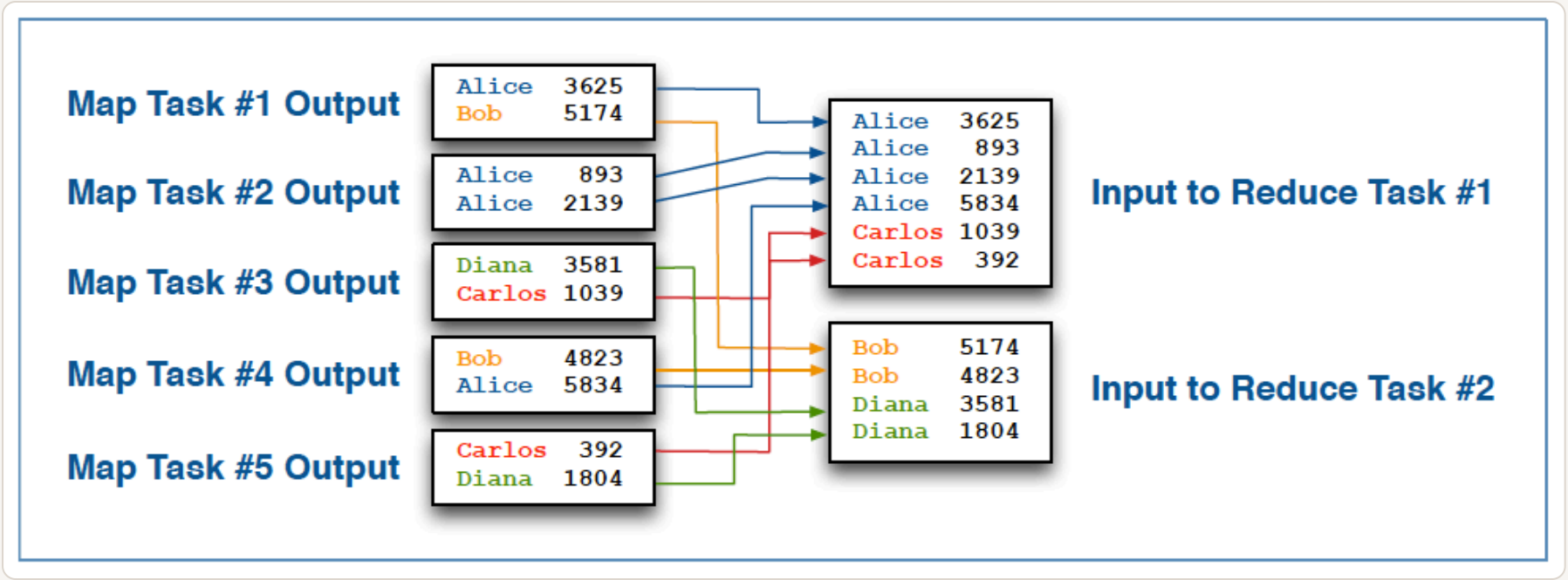


Map phase

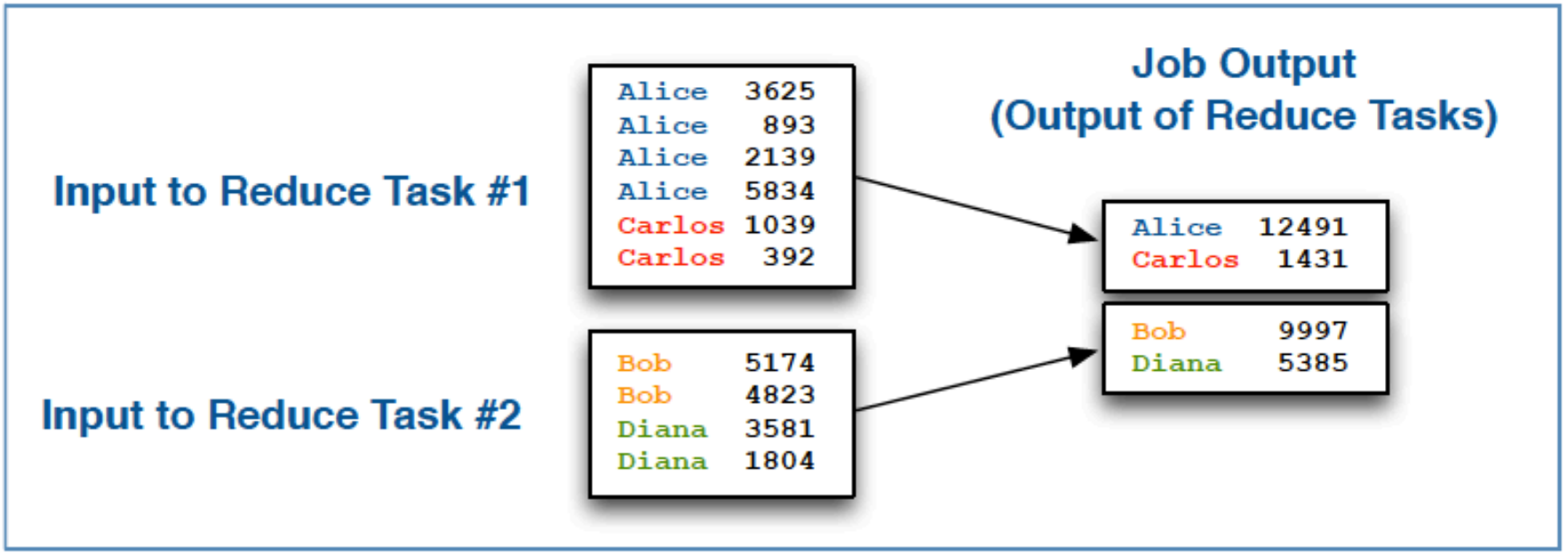

Job Input
0 Alice 3625
1 Bob 5174
2 Alice 893
3 Alice 2139
4 Diana 3581
5 Carlos 1039
6 Bob 4823
7 Alice 5834
8 Carlos 392
9 Diana 1804
Alice 3625
Bob 5174
Alice 893
Alice 2139
Diana 3581
Carlos 1039
Bob 4823
Alice 5834
Carlos 392
Diana 1804
Output from Map Tasks

Shuffle & sort

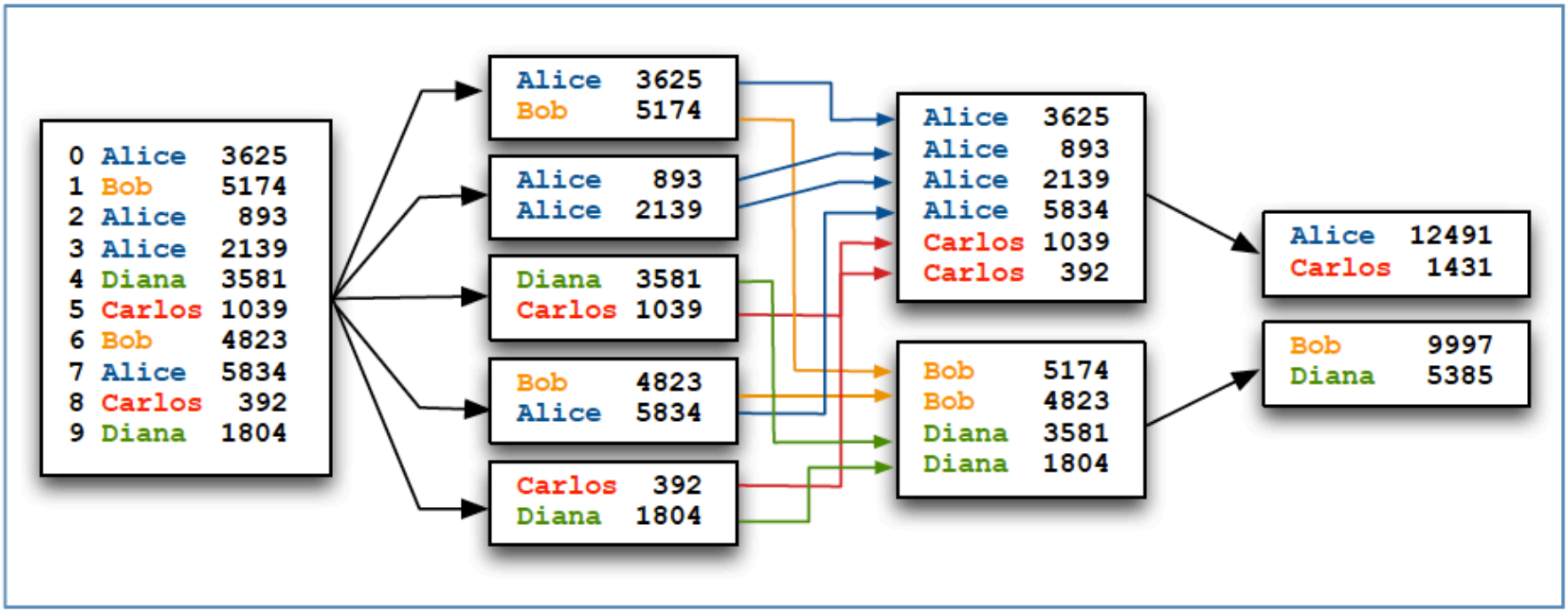
Framework groups intermediate keys into partitions for reducers.



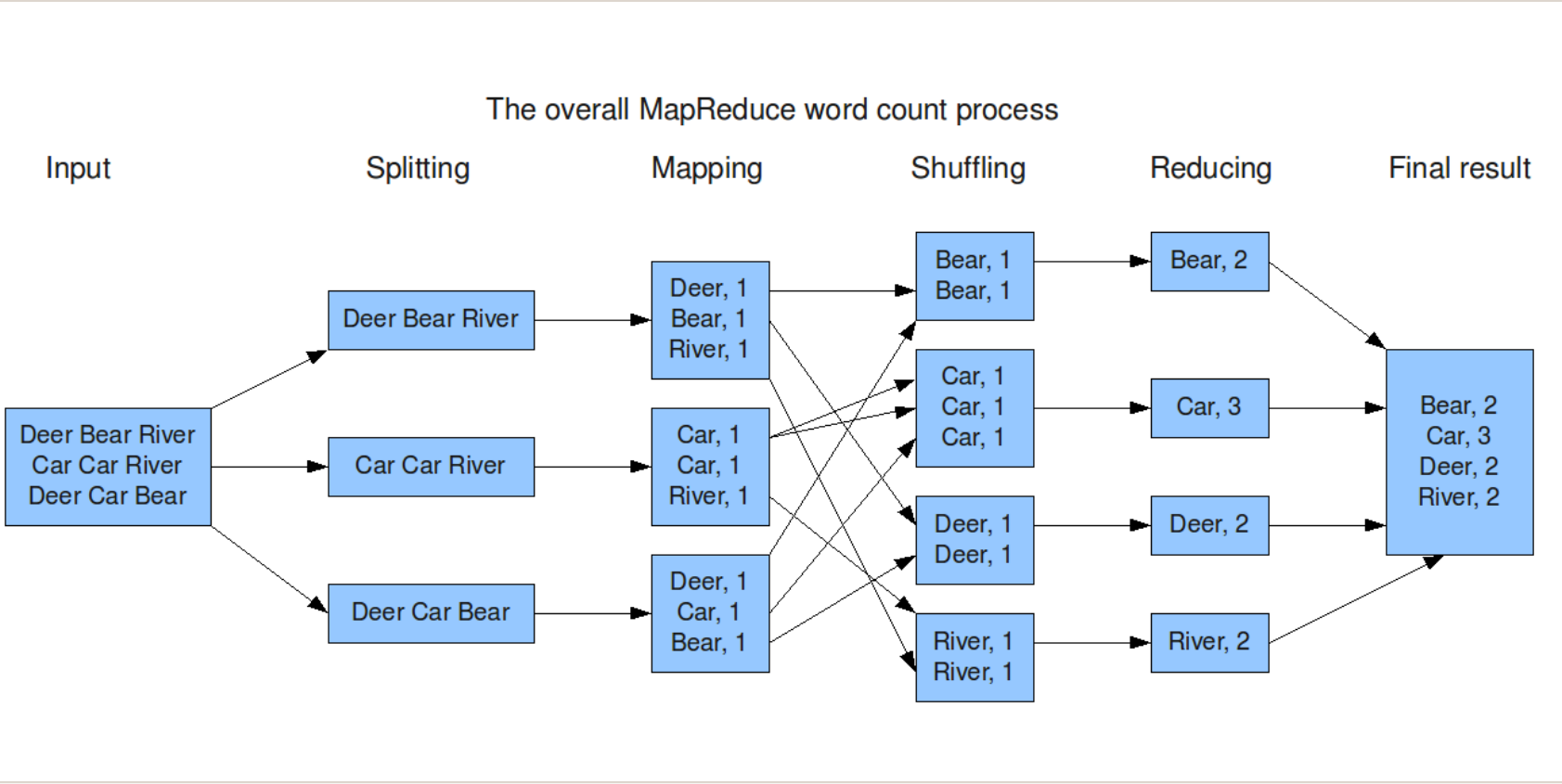
Reduce phase



End-to-end dataflow



WordCount dataflow



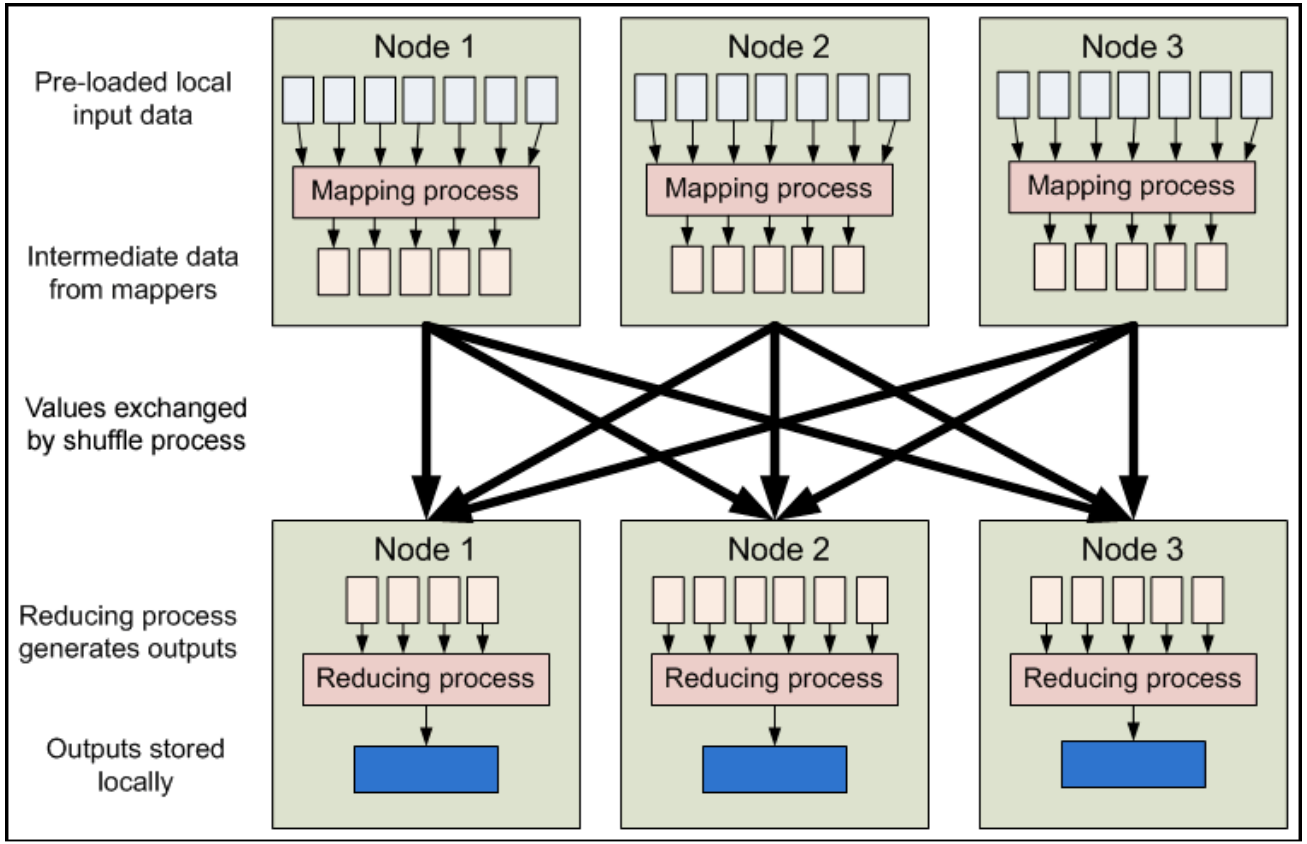
WordCount code (1)

```
9  import org.apache.hadoop.mapreduce.Job;
10 import org.apache.hadoop.mapreduce.Mapper;
11 import org.apache.hadoop.mapreduce.Reducer;
12 import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
13 import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
14 import org.apache.hadoop.util.GenericOptionsParser;
15
16
17
18
19 public class WordCount {
20 public static void main(String [] args) throws Exception
21 {
22 Configuration c=new Configuration();
23 String[] files=new GenericOptionsParser(c,args).getRemainingArgs();
24 Path input=new Path(files[0]);
25 Path output=new Path(files[1]);
26 Job j=new Job(c,"wordcount");
27 j.setJarByClass(WordCount.class);
28 j.setMapperClass(MapForWordCount.class);
29 j.setReducerClass(ReduceForWordCount.class);
30 j.setOutputKeyClass(Text.class);
31 j.setOutputValueClass(IntWritable.class);
32 FileInputFormat.addInputPath(j, input);
33 FileOutputFormat.setOutputPath(j, output);
34 System.exit(j.waitForCompletion(true)?0:1);
35 }
```

WordCount code (2)

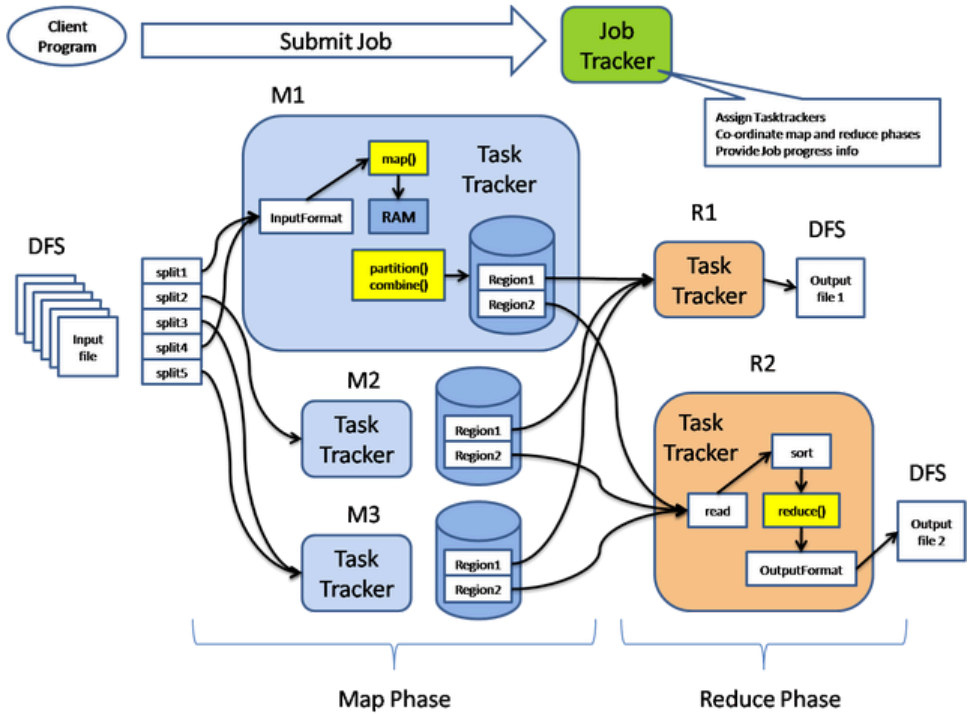
```
36 public static class MapForWordCount extends Mapper<LongWritable, Text, Text, IntWritable>{
37 public void map(LongWritable key, Text value, Context con) throws IOException, InterruptedException
38 {
39 String line = value.toString();
40 String[] words=line.split(",");
41 for(String word: words )
42 {
43       Text outputKey = new Text(word.toUpperCase().trim());
44   IntWritable outputValue = new IntWritable(1);
45   con.write(outputKey, outputValue);
46 }
47 }
48 }
49
50 public static class ReduceForWordCount extends Reducer<Text, IntWritable, Text, IntWritable>
51 {
52 public void reduce(Text word, Iterable<IntWritable> values, Context con) throws IOException, InterruptedE
53 {
54 int sum = 0;
55    for(IntWritable value : values)
56    {
57    sum += value.get();
58    }
59    con.write(word, new IntWritable(sum));
60 }
```

MapReduce on a cluster



JobTracker & TaskTracker (MR1)

Classic Hadoop 1.x roles — later folded into **YARN** application model.

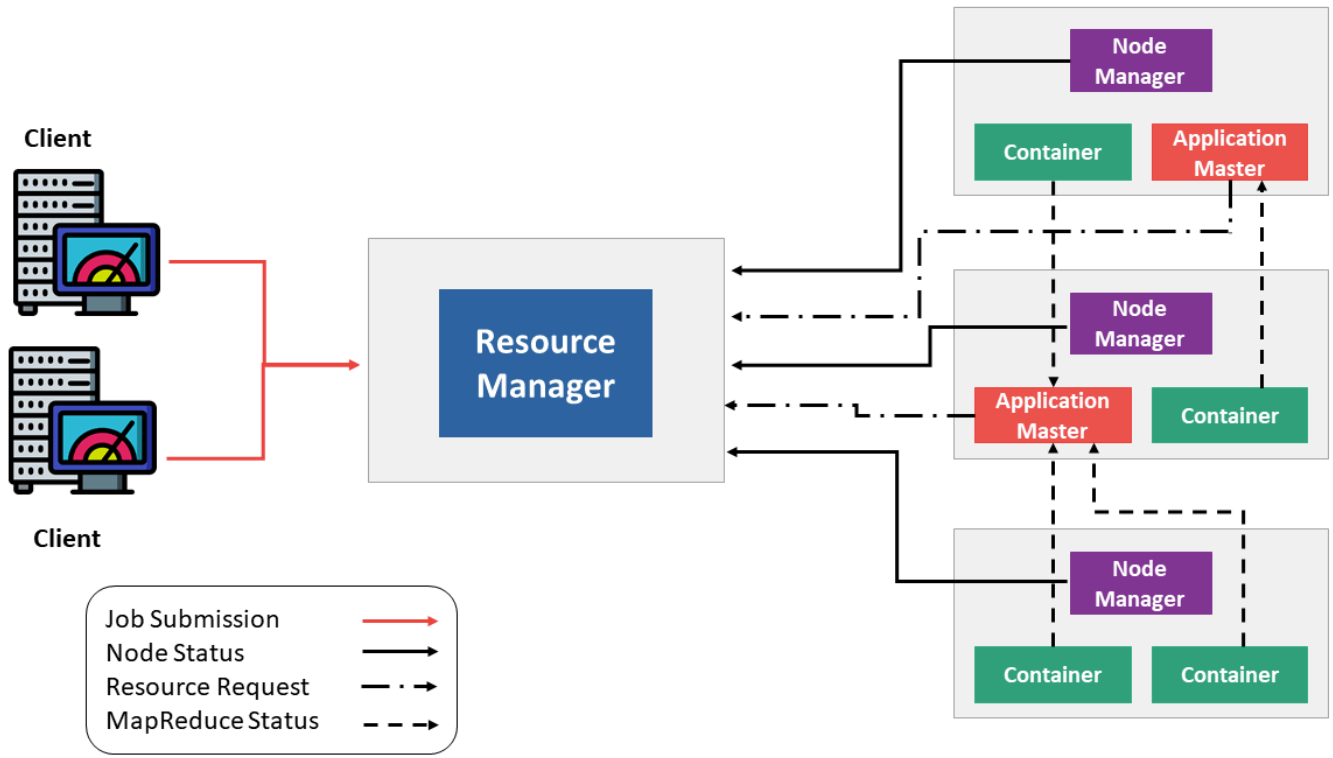


YARN — resource negotiation

- Nodes offer **CPU + memory**
- YARN allocates containers to applications
- Hadoop 2+: MapReduce **and** other engines share one cluster
- JobTracker duties → per-app **ApplicationMaster**

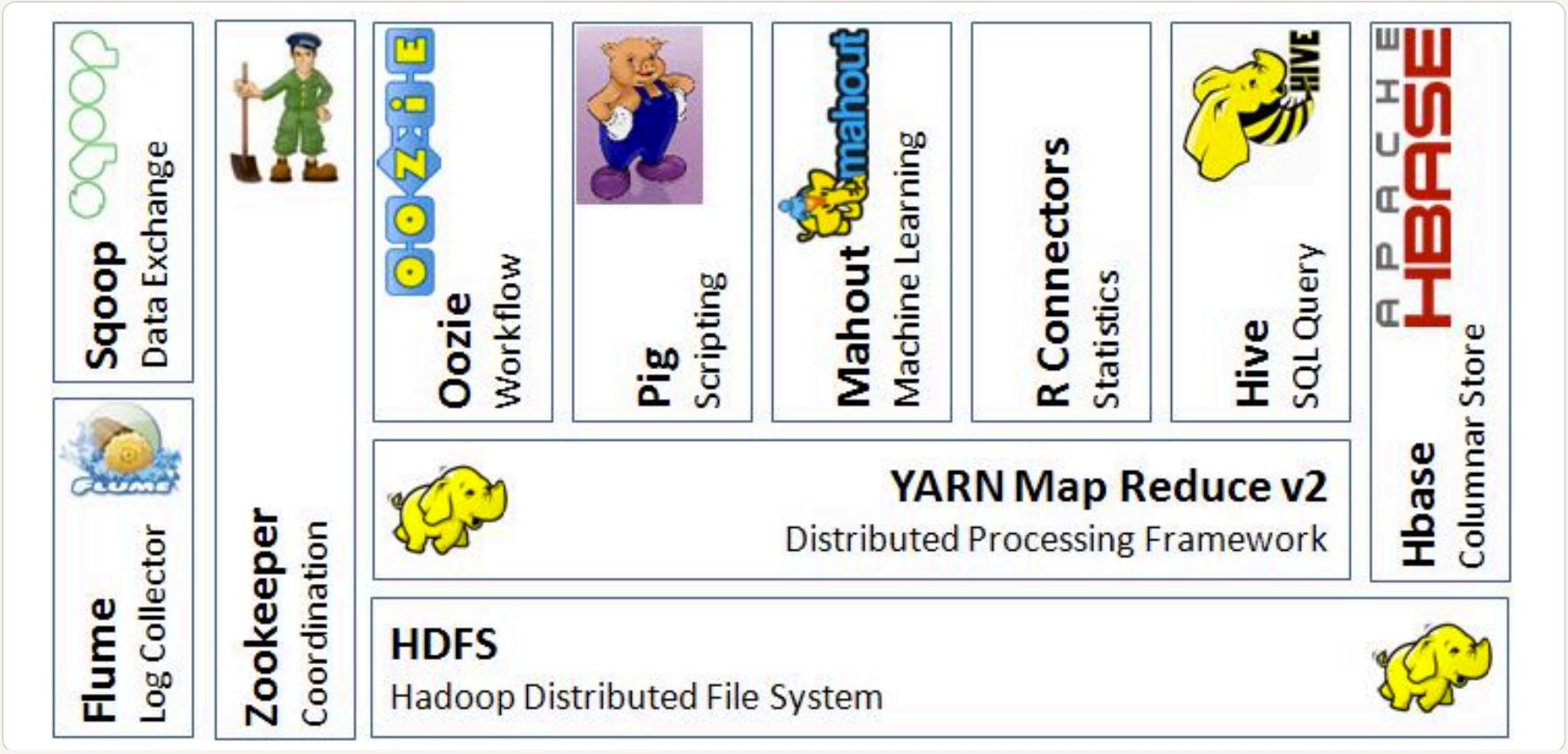
```
Hadoop 1: HDFS + MapReduce (resource + processing coupled)
Hadoop 2: HDFS + YARN + MapReduce / Spark / …
```

YARN allocation example



Ecosystem around the core

Big picture ecosystem



Apache Pig

What	High-level **dataflow** language (Pig Latin) over Hadoop
Why	Faster to write joins / transforms than raw MapReduce Java
How	Compiles to MapReduce (historically) / Tez jobs
2026	Mostly **legacy** — Spark SQL / DataFrame cover the same niche

Use Pig to understand the **abstraction ladder**: MR → scripting → SQL engines.

Apache Hive

What	SQL-like **HiveQL** warehouse layer on Hadoop storage
Why	Analysts write SQL instead of MapReduce code
How	Plans run on MR / Tez / **Spark** engines
2026	Still seen in lakes; often replaced/paired with Spark SQL, Trino (L07)

Hive popularized **SQL on HDFS** — the idea lives on in lakehouse query engines.

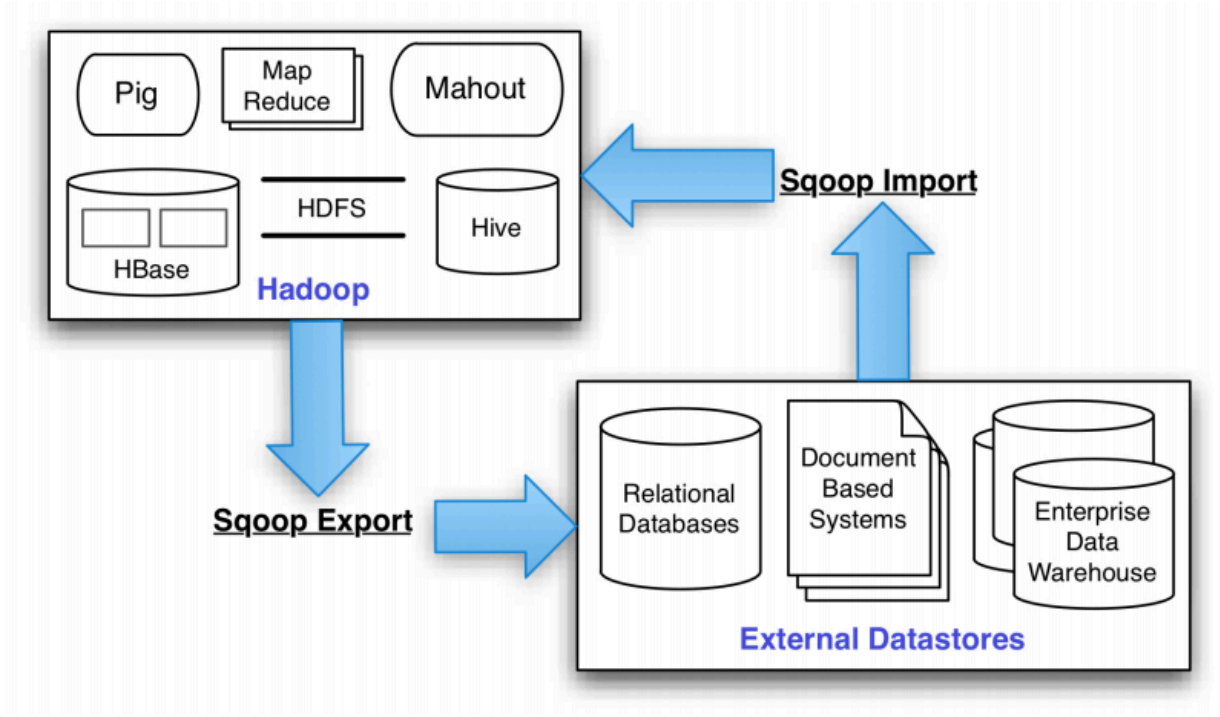
Apache HBase

What	Wide-column **NoSQL** store on HDFS
Why	Random get/put + high write rates (unlike HDFS append/scan style)
API	get / put / scan — not full SQL
Later	NoSQL families & Dynamo-style designs → **L05–L06**

HBase fills the gap when you need **serving / mutable** access on the same cluster storage.

Apache Sqoop

Bulk transfer between Hadoop and relational DBs (import/export via Map-heavy jobs).



Kafka in the picture

- Decouples producers and consumers
- Reliable buffered transport between systems
- Not “core Hadoop”, but standard in the broader stack → **L08**

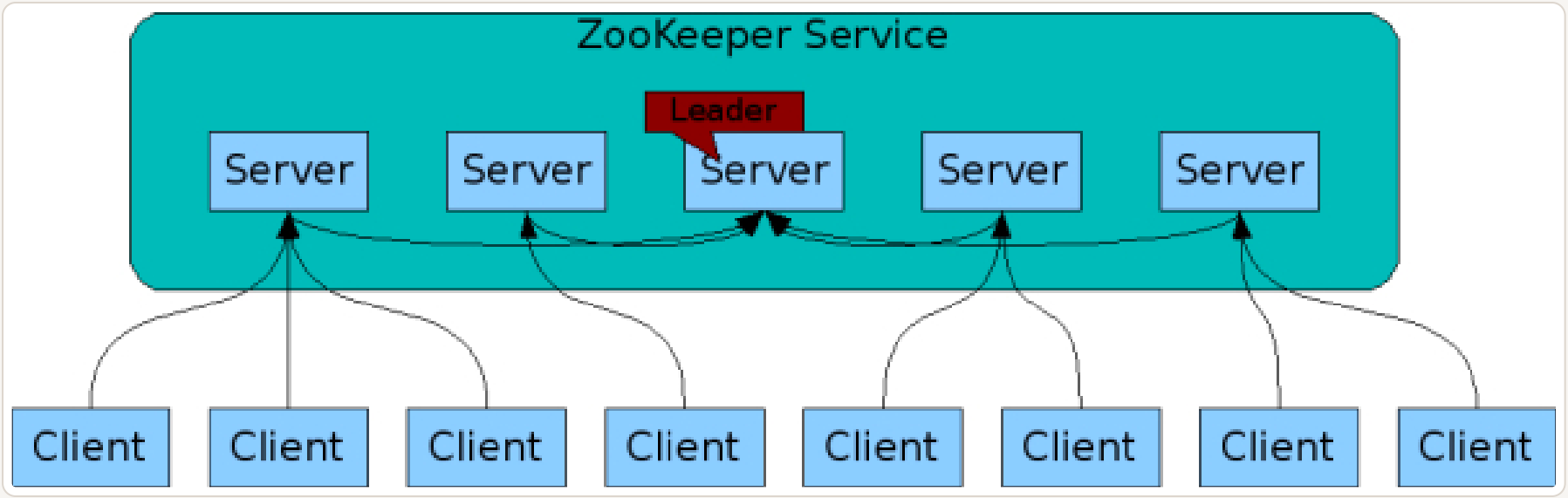
Apache Oozie

What	Workflow **scheduler** for Hadoop jobs (DAG of actions)
Actions	MR / Pig / Hive / Java / shell / HDFS / SSH / email
Why	Chain multi-step batch pipelines with dependencies
2026	Airflow / cloud schedulers dominate new work — same **DAG** idea → **L14**

Oozie teaches **orchestration as a control plane** separate from the data plane.

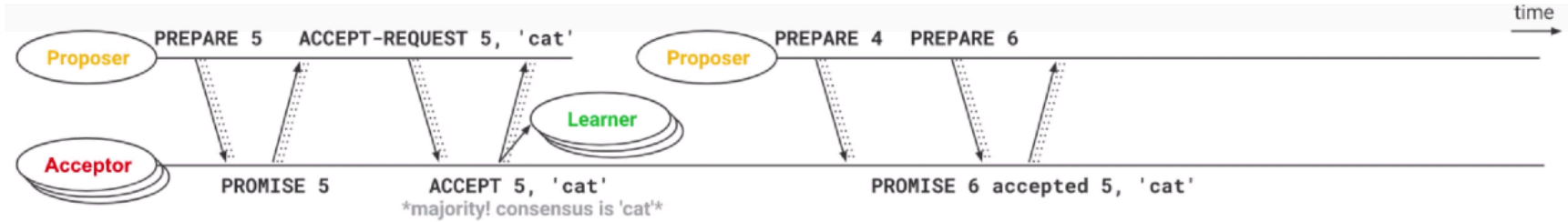
Apache ZooKeeper

Coordination: membership, leader election, dynamic config, health — building block for distributed systems.



Consensus aside: Paxos

Coordination services rely on consensus algorithms (ZooKeeper-style).



⇨ Proposer wants to propose a certain value:
It sends **PREPARE IDp** to a majority (or all) of **Acceptors**.
IDp must be unique, e.g. slotted timestamp in nanoseconds.
e.g. Proposer 1 chooses IDs 1, 3, 5...
Proposer 2 chooses IDs 2, 4, 6..., etc.
Timeout? retry with a new (higher) IDp.

⇨ **Acceptor** receives a **PREPARE** message for IDp:
Did it promise to ignore requests with this IDp?
Yes -> then ignore
No -> Will promise to ignore any request lower than IDp.
Has it ever accepted anything? (assume accepted ID=**IDa**)
Yes ->Reply with **PROMISE IDp accepted IDa, value**.
No -> Reply with **PROMISE IDp**.

1 If a majority of acceptors promise, no ID<IDp can make it through.

⇨ Proposer gets majority of **PROMISE** messages for a specific IDp:
It sends **ACCEPT-REQUEST IDp, VALUE** to a majority (or all) of **Acceptors**.
Has it got any already accepted value from promises?
Yes - > It picks the value with the highest **IDa** that it got.
No -> It picks any value it wants.

⇨ **Acceptor** receives an **ACCEPT-REQUEST** message for IDp, value:
Did it promise to ignore requests with this IDp?
Yes -> then ignore
No -> Reply with **ACCEPT IDp, value**. Also send it to all **Learners**.

2 If a majority of acceptors accept IDp, value, consensus is reached.
Consensus is and will always be on value (not necessarily IDp).

⇨ Proposer or Learner get **ACCEPT** messages for IDp, value:

3 If a proposer/learner gets majority of accept for a specific IDp,
they know that consensus has been reached on value (not IDp).

From Hadoop to Spark (thinking model)

- Same goals: **compute near data**, fault tolerance, scale-out
- Spark: richer DAG, in-memory, fewer MR job barriers
- Many Hive/Pig workloads moved to Spark SQL / DataFrame APIs
- You still need the **storage & locality** intuition from HDFS/YARN era

2.3 Hadoop on clouds

Managed platforms

Vendor logos change fast — think in **categories**, not brands:

Category	Role	Examples (illustrative)
Managed Hadoop/Spark	Cluster + YARN/Spark as a service	EMR, Dataproc, HDInsight
Object storage	Durable bytes (often replaces bare HDFS)	S3, ADLS, GCS
Query / warehouse	SQL over lake or warehouse	Athena, BigQuery, Snowflake, Trino
Streaming	Event backbone	Kafka / MSK, Pulsar, Kinesis

Same principles: **partition, replicate, scale-out, fail & retry**.

Cloud practice (2026 framing)

Pattern	Typical pieces
Managed Hadoop/Spark	EMR, HDInsight, Dataproc, …
Storage	Often **object store** (S3/ADLS/GCS) instead of only HDFS
Compute	Spark jobs; MR rare for new work
Same principles	Partitioning, fault tolerance, scale-out, locality/cost tradeoffs

Link to the course

Next	What you deepen
L03	HDFS internals
L04	Lakehouse on object storage
L05–L06	NoSQL · DynamoDB (HBase-class thinking)
L07	Presto / Trino (SQL federation)
L08	Kafka
L09–L12	Architectures · Spark · Flink
L13–L14	Spark ML · end-to-end lakehouse pipeline

Takeaways

1. Hadoop = **scale-out storage + batch processing** on commodity clusters
2. **HDFS / MapReduce / YARN** are the conceptual core
3. MapReduce = **map → shuffle → reduce** over key-value pairs
4. Ecosystem tools raise abstraction (Hive/Pig) or fill gaps (HBase/Sqoop/Oozie/ZK)
5. In 2026, treat Hadoop as **foundation + legacy**; Spark/cloud keep the principles

Next

L03 — HDFS